

IPPL Python Interoperability Layer

Nikola Mamić

JSC
Guest Student Programme

September 29, 2026

The idea

- A general interoperability layer between Python and IPPL
- Keeps the data on the same device on both ends (minimize copy)
- Ease of use
- Fully opt-in, header-only (pybind11, DLPack)



C++ miniapp

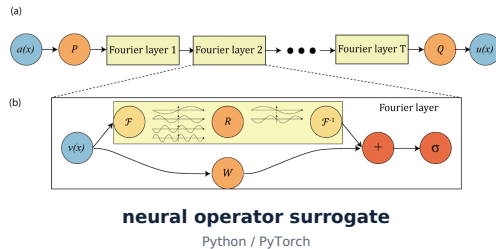
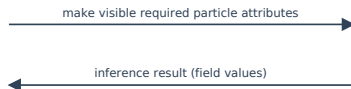


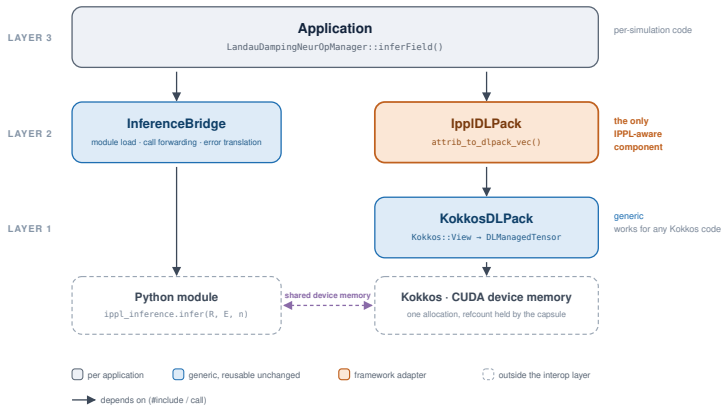
Figure: The interaction between IPPL and the opt-in surrogate written in Python

The idea

- 1 Kokkos View $\rightarrow$ DLPack Python capsule
- 2 IPPL $\rightarrow$ DLPack (defers to 1)
- 3 Bridge between Python interpreter and C++ interface
- 4 Manager + miniapp

IPPL to Python interoperability layers

Application builds DLPack capsules and passes them through the bridge.



- Translation table between Kokkos Views and DLPack capsules
- Capsule holds reference to View, keeping the allocation alive while Python uses it
- **Agnostic to everything besides DLPack and Kokkos**

Kokkos	DLPack
memory_space	DLDevice
value_type	DLDataType
extent(i)	shape[i]
stride(i)	strides[i]

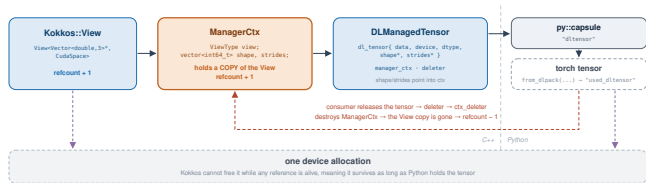


Figure: The lifetime/ownership of the View allocation

```
template <typename ManagedT, typename ScalarT, typename ViewType>
py::capsule export_raw(ViewType view, ScalarT* scalar_ptr,
                        std::vector<int64_t> shape, std::vector<int64_t> strides,
                        ReadOnly read_only) {
    using MemorySpace = typename ViewType::memory_space;

    DLDevice device = device_of<MemorySpace>();
    DLDataType dtype = dtype_of<ScalarT>();

    auto* ctx = new ManagerCtx<ViewType>(view, std::move(shape), std::move(strides));
    auto* managed = new ManagedT();

    // DLTensor is the plain C Tensor object
    managed->dl_tensor.data = static_cast<void*>(scalar_ptr); // Data ptr to allocated data
    managed->dl_tensor.device = device;
    managed->dl_tensor.ndim = static_cast<int32_t>(ctx->shape.size());
    managed->dl_tensor.dtype = dtype; //Data type of the pointer
    managed->dl_tensor.shape = ctx->shape.data();
    managed->dl_tensor.strides = ctx->strides.data();
    managed->dl_tensor.byte_offset = 0; // offset in struct to the beginning pointer to data in bytes
    managed->manager_ctx = ctx;
    managed->deleter = &ctx_deleter<ManagedT, ViewType>;
}
```

Figure: Code snippet doing the mapping

- Converts IPPL attribute (AoS) to a DLPack capsule
- "Peels" down to array of primitive scalars, which DLPack understands
- Bytes don't change, only their description does
- Zero-copy when value type is standard layout, device-copy fallback otherwise

```
namespace py = pybind11;

template <typename Attrib>
py::capsule attrib_to_dlpack_vec(Attrib& attrib, kokkos_dlpack::ReadOnly ro,
                                kokkos_dlpack::DLPackVersion ver) {
    // peeling the onion - say we are given a ippl::ParticleAttrib<ippl::Vector<double,3>>:
    using view_type = typename Attrib::view_type; // the underlying kokkos view, e.g. Kokkos::View<Vector<double,3>*>
    using vector_type = typename view_type::value_type; // elements of the kokkos view, e.g. ippl::Vector<double,3>
    using ScalarT = typename vector_type::value_type; // the scalar elements themselves, e.g. double
    constexpr int64_t Dim = static_cast<int64_t>(vector_type::dim); // the shape of the data, e.g. 3

    static_assert(view_type::rank == 1, "ParticleAttrib is always rank-1");

    // gets the attribute Kokkos View, trimmed to live particles
    view_type v = attrib.getView();
    // ...and their count, cast from size_t to int64_t, which DLPack uses
    // (ParticleAttrib is always rank-1 View, so extent(0) gets the count)
    const int64_t N = static_cast<int64_t>(v.extent(0));
```

Figure: Code snippet retrieving the needed attributes

- Loads a Python module by name at runtime, chosen by configuration
- Single variadic entry point, handles any function name and any arguments pybind11 can convert
- Failures contained and guarded against, a Python exception is a collective abort

```
class InferenceBridge {
private:
    py::module_ mod_;

    template <typename F>
    auto guarded(F&& f) -> decltype(f()) { ...

public:
    InferenceBridge(const std::string& dir, const std::string& name) {
        guarded([&]{
            py::module_::import("sys").attr("path").attr("insert")(0, dir);
            mod_ = py::module_::import(name.c_str());
        });

        //Generic variadic call function, pybind converts params to Python objects
        // and py::object::operator() is itself a variadic forwarding template
        template <typename... Args>
        py::object call(const char* fn, Args&&... args) {
            return guarded([&]{return mod_.attr(fn)(std::forward<Args>(args)...); });
        }

        void finalize() {
            guarded([&]{ mod_.attr("finalize")(); });
            mod_ = py::object();
        }
    };
};
```

Figure: The entire InferenceBridge class

The Application (Landau Damping)

- Minimal changes needed to use the surrogate
- No modification to IPPL part
- Thin API
- Black box approach to surrogate from IPPL's standpoint

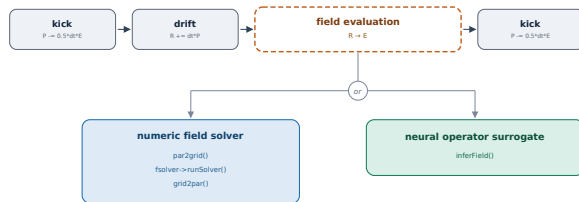


Figure: The interchangeability of the solver and surrogate

The Managers (Landau Damping)

```
void LeapFrogStep() {
    IpplTimings::startTimer(PTimer);
    pc->P = pc->P - 0.5 * dt * pc->E;
    IpplTimings::stopTimer(PTimer);

    // drift
    IpplTimings::startTimer(RTimer);
    pc->R = pc->R + dt * pc->P;
    IpplTimings::stopTimer(RTimer);

    // Since the particles have moved spatially update them to correct processors
    IpplTimings::startTimer(updateTimer);
    pc->update();
    IpplTimings::stopTimer(updateTimer);

    size_type totalP      = this->totalP_m;
    int it                = this->it_m;
    bool isFirstRepartition = false;
    if (this->loadbalancer_m->balance(totalP, it + 1)) {-

        // scatter the charge onto the underlying grid
        this->par2grid();

        // Field solve
        IpplTimings::startTimer(SolveTimer);
        this->fsolver_m->runSolver();
        IpplTimings::stopTimer(SolveTimer);

        // gather E field
        this->grid2par();

        // kick
        IpplTimings::startTimer(PTimer);
        pc->P = pc->P - 0.5 * dt * pc->E;
        IpplTimings::stopTimer(PTimer);
    }
}
```

Figure: LandauDampingManager.h
LeapFrogStep, called in advance()

```
void advance() override {
    if (this->stepMethod_m != "LeapFrog")
        throw IpplException("advance()", "Step method not set/Not implemented");

    double dt = this->dt_m;
    auto pc = this->pcontainer_m;
    auto fc = this->fcontainer_m;

    pc->P = pc->P - 0.5 * dt * pc->E;      // kick
    pc->R = pc->R + dt * pc->P;           // drift

    // Apply the periodic BC only to keep nlocal constant for the whole run.
    auto& layout = pc->getLayout();
    layout.applyBC(pc->R, layout.getRegionLayout().getDomain()); // just the periodic BC

    inferField(); //replaced par2grid, pic solver, then grid2par

    pc->P = pc->P - 0.5 * dt * pc->E;
}
```

Figure: LandauDampingNeurOpManager.h
advance()

The Inference Manager (Landau Damping)

- Exposes the ParticleAttributes to the Python side as capsules, converting via IpplDLPack.h
- Calls function in the Python script over the interpreter using InferenceBridge.h, passing the capsules and rank number as parameters
- Python inference writes result directly into E

```
void inferField() {  
    static IpplTimings::TimerRef inferTimer = IpplTimings::getTimer("inference");  
  
    auto pc = this->pcontainer_m;  
    Kokkos::fence();  
  
    py::capsule R_cap = ippl_dlpack::attrib_to_dlpack_vec(  
        pc->R, kokkos_dlpack::ReadOnly::Yes, kokkos_dlpack::DLPackVersion::Legacy);  
  
    py::capsule E_cap = ippl_dlpack::attrib_to_dlpack_vec(  
        pc->E, kokkos_dlpack::ReadOnly::No, kokkos_dlpack::DLPackVersion::Legacy);  
  
    IpplTimings::startTimer(inferTimer);  
    bridge->call("infer", R_cap, E_cap, pc->getLocalNum());  
    Kokkos::fence();  
    IpplTimings::stopTimer(inferTimer);  
}
```

Figure: inferField() in
LandauDampingNeurOpManager.h

```
@torch.no_grad()  
def infer(R_cap, E_cap, nlocal):  
    t0 = time.perf_counter()  
    R = from_dlpack(R_cap).T # (dim, N_local) float64, this is transposed from IPPL  
    # which saves as (N_local, dim)  
    E = from_dlpack(E_cap).T #the write target
```

Figure: infer() in ippl_inference.py

LandauDamping.cpp vs LandauDampingNeurOp.cpp

```
// Create an instance of a manager for the considered application
LandauDampingManager<T, Dim> manager(totalP, nt, nr, lbt, solver, step_method,
| | | | | | | | | | preconditioner_params);

// Perform pre-run operations, including creating mesh, particles,...
manager.pre_run();

IpplTimings::stopTimer(initializeTimer);

manager.setTime(0.0);

msg << "Starting iterations ..." << endl;

manager.run(manager.getNt());

msg << "End." << endl;

IpplTimings::stopTimer(mainTimer);
IpplTimings::print();
IpplTimings::print(std::string("timing.dat"));
```

Figure: LandauDamping.cpp

```
const char* infer_dir = std::getenv("IPPL_INFER_DIR");
if (!infer_dir) {
    throw std::runtime_error("IPPL_INFER_DIR not set");
}

py::scoped_interpreter guard{};

InferenceBridge bridge(infer_dir, "ippl_inference");
LandauDampingNeurOpManager<T, Dim> manager(totalP, nt, nr, lbt, solver,
| | | | | | | | | | step_method, preconditioner_params, &bridge);
manager.pre_run();

IpplTimings::stopTimer(initializeTimer);

manager.setTime(0.0);
manager.run(manager.getNt());
bridge.finalize();

IpplTimings::stopTimer(mainTimer);
IpplTimings::print();
IpplTimings::print(std::string("timing.dat"));
```

Figure: LandauDampingNeurOp.cpp

Showcase: Inference Scaling Run Through IPPL

- Scaling runs for inference using the neural operator model w/ NUFFT
- 400 timesteps, 1-512 GPUs, up to 1.6B particles
- Done using the IPPL/Python interoperability layer

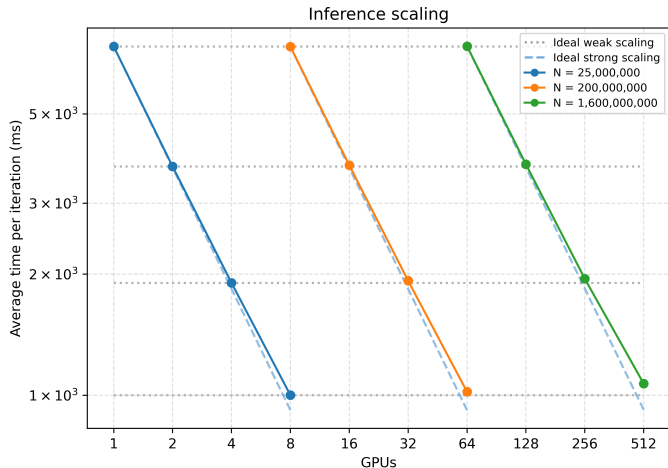


Figure: Strong and weak scaling

Showcase: Inference Scaling Run Through IPPL

- Efficiency = T_{ideal}/T : how long a run should take over how long it did. $E = 1$ is perfect, $E < 1$ is lost time.
- **Strong scaling** (solid, one curve per N): N fixed, twice the GPUs should halve the time.
$$E_s(P) = P_0 \cdot T(P_0) / (P \cdot T(P))$$
- **Weak scaling** (dashed, one curve per load level): N/P fixed, so the time should not change.
$$E_w(P) = T_{\text{base}} / T(P).$$

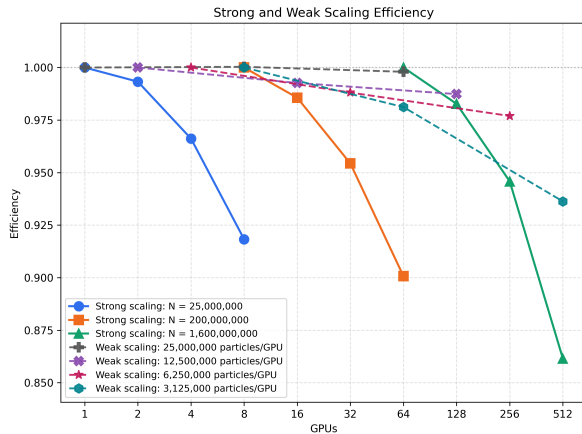


Figure: Efficiency of scaling

Showcase: Inference Scaling Run Through IPPL

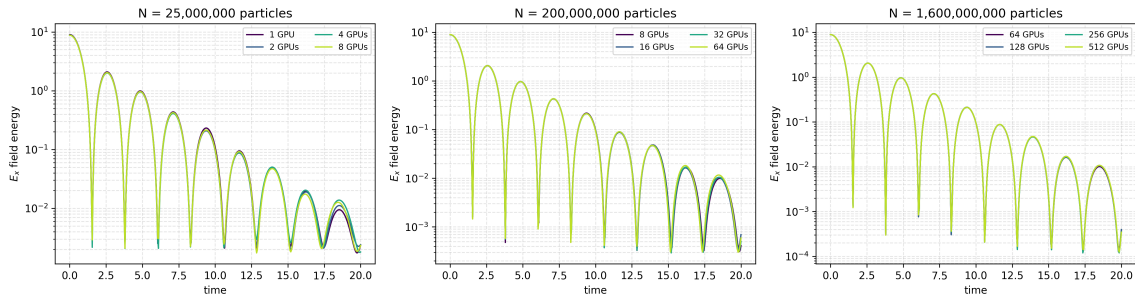


Figure: The results of different GPU counts

- IPPL gets interoperability with PyTorch, CuPy, JAX
- Enables use of completely separately developed Python models/tools
- No changes needed in IPPL code, no interference with existing functionalities
- Opt-in, header-only